



Christophe PICHAUD  
Architecte Microsoft chez  
'M Applications by Devoteam'  
christophepichaud@hotmail.com  
www.windowscpp.com

# Détection faciale et reconnaissance faciale avec OpenCV4 en C++

*Les services cognitifs ont le vent en poupe et la détection des visages et leur reconnaissance est un sujet à la mode. Il existe des services comme Azure Cognitive Services et Azure Computer Vision mais aussi des services open-source donc gratuits... à faire tourner en local sans passer par le cloud. On peut aussi y mixer du machine learning et de l'IA. C'est ce que nous allons mixer dans l'article de ce mois-ci.*

## Introduction à OpenCV

Créée en 2000 par Intel, la librairie OpenCV (Open Source Computer Vision) est une bibliothèque C/C++ temps réel pour le traitement des images. La documentation et les packages Windows, Linux, Mac sont disponibles sur [opencv.org](http://opencv.org). Cette bibliothèque est leader dans son domaine. Elle utilise massivement la STL (Standard Template Library) du C++. Il existe aussi des bindings pour Python, Java, Haskell, Perl, Ruby. Et aussi une version hybride EMGU pour .NET. Il existe aussi deux modes d'accélération matérielle :

- CUDA ;
- OpenCL.

## Opérations de bases

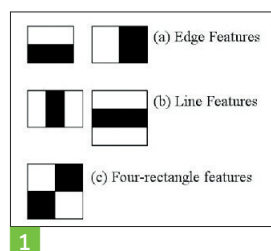
La gestion des images requiert des classes particulières. Le namespace `cv` contient de nombreuses classes C++ :

- `Scalar` pour la couleur ;
- `Rect`, `Point`, `Size` ;
- `Mat` pour les images.

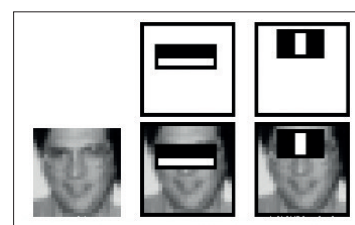
## Détection de visages via Cascades Haar

Commençons par la détection de visages. La détection d'objets à l'aide des classificateurs en cascade basés sur des fonctionnalités Haar est une méthode de détection d'objets efficace proposée par Paul Viola et Michael Jones dans leur article, « Détection rapide d'objets utilisant une cascade boostée de fonctionnalités simples » en 2001. C'est une approche basée sur l'apprentissage par machine où une fonction cascade est formée à partir de beaucoup d'images positives et négatives. Elle est ensuite utilisée pour détecter des objets dans d'autres images. Ici, nous allons travailler avec la détection de visages. Initialement, l'algorithme a besoin de beaucoup d'images positives (images de visages) et d'images négatives (images sans visages) pour former le classificateur. Ensuite, nous avons besoin d'extraire des fonctionnalités de celui-ci. Pour cela, les fonctions Haar affichées dans l'image ci-dessous sont utilisées. Ils sont comme notre noyau à convolution. Chaque fonction est une valeur unique obtenue en soustrayant la somme des pixels sous le rectangle blanc de la somme des pixels sous le rectangle noir. **1**

Maintenant, toutes les tailles et les emplacements possibles de chaque noyau sont employés pour calculer beaucoup de dispositifs. Imaginez à quel point il y a besoin de calcul ! Même une fenêtre



1



2

24x24 donne des résultats de plus de 160000 fonctionnalités). Pour chaque calcul de fonction, nous devons trouver la somme des pixels sous les rectangles blancs et noirs. Pour résoudre ce fait, ils ont introduit l'image intégrale. Quelle que soit la taille de votre image, elle réduit les calculs d'un pixel donné à une opération impliquant seulement quatre pixels. Bien, n'est-ce pas ? Ça rend les choses super rapides.

Mais parmi toutes ces caractéristiques, nous avons calculé : la plupart d'entre eux sont hors de propos. Par exemple, considérez l'image ci-dessous. La rangée du haut montre deux bonnes caractéristiques. La première caractéristique choisie semble se concentrer sur la propriété que la région des yeux est souvent plus sombre que la région du nez et des joues. La deuxième caractéristique choisie repose sur la propriété que les yeux sont plus foncés que le pont du nez. Mais les mêmes fenêtres appliquées aux joues ou à tout autre endroit ne sont pas pertinentes. Alors, comment pouvons-nous choisir les meilleures caractéristiques de 160000 et + caractéristiques ? C'est réalisé par AdaBoost. **2**

Pour cela, nous appliquons chaque fonctionnalité sur toutes les images de la formation. Pour chaque fonctionnalité, AdaBoost trouve le meilleur seuil qui classe les faces positives et négatives. Évidemment, il y aura des erreurs ou des erreurs de classification. Nous sélectionnons les fonctionnalités avec des taux d'erreur minimum, ce qui signifie qu'elles sont les caractéristiques qui classent plus précisément le visage et les autres images. Néanmoins, le processus n'est pas aussi simple que cela. Chaque image se voit attribuée un poids égal au début. Après chaque classification, le poids des images mal classées sont augmentés. Alors le même processus est refait. De nouveaux taux d'erreurs sont calculés. Également de nouveaux poids. Le processus se poursuit jusqu'à ce que le taux

d'exactitude ou d'erreur requis soit atteint ou le nombre requis de fonctionnalités sont trouvées.

Le dernier classificateur correspond à une somme pondérée de ces faibles classificateurs. Elle est qualifiée de faible parce que seul, il ne peut pas classer l'image, mais avec d'autres, il forme un classificateur fort. La documentation dit même que 200 fonctionnalités fournissent la détection avec une précision de 95 %. Leur configuration finale avait environ 6000 caractéristiques. (Imaginez une réduction de 160000 + caractéristiques à 6000 caractéristiques. C'est un gros gain).

Alors maintenant, vous prenez une image. Prendre chaque fenêtre 24 x 24. Appliquez-lui 6000 caractéristiques. Vérifiez si c'est le visage ou pas. Wow... N'est-il pas un peu inefficace et une perte de beaucoup de temps ? Oui. Les auteurs de OpenCV ont une bonne solution pour cela.

Dans une image, la plupart de l'image n'est pas la région du visage. Aussi est-il préférable d'avoir une méthode simple pour vérifier si une fenêtre n'est pas une région du visage. Si ce n'est pas le cas, jetez-le en un seul coup et il ne sera pas traité à nouveau. Au lieu de cela, se concentrer sur les régions où il peut y avoir un visage. De cette façon, nous avons passé plus de temps à vérifier les régions du visage possibles.

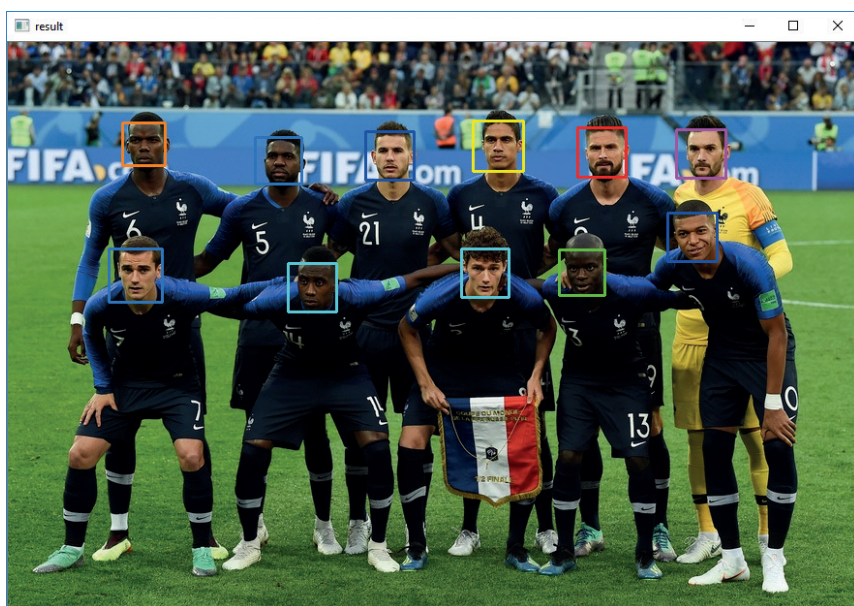
Pour cela, ils ont introduit le concept de **Cascade de classificateurs**. Au lieu d'appliquer toutes les 6000 fonctionnalités sur une fenêtre, les fonctions sont regroupées en différents stades des classificateurs et les appliquent un par un (normalement les premières étapes contiennent beaucoup moins de fonctionnalités). Si une fenêtre ne parvient pas à la première étape, jetez-la. Nous ne considérons pas les caractéristiques restantes à ce sujet. Si elle passe, appliquer la deuxième étape de fonctionnalités et ainsi de suite. La fenêtre qui passe toutes les étapes est une région du visage. Voilà le plan !

## Codage de la détection

Il suffit de charger une image en mémoire et d'utiliser une routine qui se nomme `CascadeClassifier::detectMultiScale`. L'utilisation de cette classe doit être faite aussi en faisant appel à `load()` en lui passant un nom de fichier de cascades. OpenCV fournit ces fichiers de données en standard. Il y en a pour le visage, les yeux, le corps, etc.

```
string cascadeName = "haarcascade_frontalface_alt.xml";
string nestedCascadeName = "haarcascade_eye_tree_eyeglasses.xml";
scale = 1.3;
if (!nestedCascade.load(samples::findFileOrKeep(nestedCascadeName)))
{
    cerr << "Could not load classifier cascade" << endl;
    return 1;
}

if (!cascade.load(samples::findFile(cascadeName)))
{
    cerr << "Could not load classifier cascade" << endl;
```



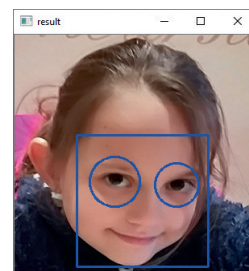
3

```
return 1;
}

double scale = 1.3;
string inputName = inputName = "image.jpg";
bool tryflip = false;

Mat image = imread(samples::findFileOrKeep(inputName), IMREAD_COLOR);
if (image.empty())
{
    if (!capture.open(samples::findFileOrKeep(inputName)))
    {
        cout << "Could not read " << inputName << endl;
        return 1;
    }
}

cout << "Detecting face(s) in " << inputName << endl;
if (!image.empty())
{
    detectAndDraw(image, cascade, nestedCascade, scale, tryflip);
    waitKey(0);
}
```



4

La routine `imread()` lit le fichier image pour le stocker dans un objet `Mat`. Ensuite la routine magique `detectAndDraw` fait le travail magique ! 3 4

Inspectons la routine `detectAndDraw()` :

```
void detectAndDraw(Mat& img, CascadeClassifier& cascade,
    CascadeClassifier& nestedCascade,
    double scale, bool tryflip)
{
    double t = 0;
    vector<Rect> faces, faces2;
```

```

const static Scalar colors[] =
{
    Scalar(255,0,0),
    Scalar(255,128,0),
    Scalar(255,255,0),
    Scalar(0,255,0),
    Scalar(0,128,255),
    Scalar(0,255,255),
    Scalar(0,0,255),
    Scalar(255,0,255)
};
Mat gray, smallImg;

cvtColor(img, gray, COLOR_BGR2GRAY);
double fx = 1 / scale;
resize(gray, smallImg, Size(), fx, fx, INTER_LINEAR_EXACT);
equalizeHist(smallImg, smallImg);

t = (double)getTickCount();
cascade.detectMultiScale(smallImg, faces,
    1.1, 2, 0
    //|CASCADE_FIND_BIGGEST_OBJECT
    //|CASCADE_DO_ROUGH_SEARCH
    | CASCADE_SCALE_IMAGE,
    Size(30, 30));
t = (double)getTickCount() - t;
printf("detection time = %g ms\n", t * 1000 / getTickFrequency());

for (size_t i = 0; i < faces.size(); i++)
{
    Rect r = faces[i];
    Mat smallImgROI;
    vector<Rect> nestedObjects;
    Point center;
    Scalar color = colors[i % 8];
    int radius;

    rectangle(img, Point(cvRound(r.x*scale), cvRound(r.y*scale)),
        Point(cvRound((r.x + r.width - 1)*scale),
            cvRound((r.y + r.height - 1)*scale)),
        color, 2, 8, 0);

    if (nestedCascade.empty())
        continue;

    smallImgROI = smallImg(r);
    nestedCascade.detectMultiScale(smallImgROI, nestedObjects,
        1.1, 2, 0
        //|CASCADE_FIND_BIGGEST_OBJECT
        //|CASCADE_DO_ROUGH_SEARCH
        //|CASCADE_DO_CANNY_PRUNING
        | CASCADE_SCALE_IMAGE,
        Size(30, 30));

    for (size_t j = 0; j < nestedObjects.size(); j++)
    {

```

```

        Rect nr = nestedObjects[j];
        center.x = cvRound((r.x + nr.x + nr.width*0.5)*scale);
        center.y = cvRound((r.y + nr.y + nr.height*0.5)*scale);
        radius = cvRound((nr.width + nr.height)*0.25*scale);
        circle(img, center, radius, color, 2, 8, 0);
    }
}

imshow("result", img);
}

```

La routine fait le job en faisant appel à CascadeClassifier.detectMultiScale pour détecter le visage et ensuite pour détecter les yeux.

## Reconnaissance faciale avec OpenCv4

Il est possible de faire trouver à qui appartient une photo donnée. Eh oui ! Pour cela, on va utiliser un module OpenCV qui est dans contrib sur Github et qui se nomme Face.

Le repository Github est ici : [https://github.com/opencv/opencv\\_contrib](https://github.com/opencv/opencv_contrib)

Dans le répertoire face, il y a du code pour reconnaître les visages suivant 3 techniques :

- Eigen faces ;
- Fisher faces ;
- Local Binary Pattern Histograms.

## Utilisation de face

Pour faire les choses dans l'état de l'art, il faut recompilier OpenCV... ou bien incorporer les classes de Face dans votre outil. Comment marche Face ? C'est très simple, il y a trois étapes :

- Générer un modèle à partir de photos d'individus : c'est l'apprentissage ou *training* ;
- Sauvegarder le modèle ou le charger ;
- Faire une prédiction en fonction d'une image quelconque.

## L'apprentissage

Il faut créer un fichier de configuration CSV dans lequel on met les data comme suit :

Chemin du fichier image ;index ;libellé.

Exemple :

```

D:\Dev\cpp\OCV\Detection\x64\Debug\images\CT1.PNG;20;Charlize
D:\Dev\cpp\OCV\Detection\x64\Debug\images\CT2.PNG;20;Charlize
D:\Dev\cpp\OCV\Detection\x64\Debug\images\CT3.PNG;20;Charlize
D:\Dev\cpp\OCV\Detection\x64\Debug\images\CT4.PNG;20;Charlize
D:\Dev\cpp\OCV\Detection\x64\Debug\images\CT5.PNG;20;Charlize
D:\Dev\cpp\OCV\Detection\x64\Debug\images\CT6.PNG;20;Charlize
D:\Dev\cpp\OCV\Detection\x64\Debug\images\CT7.PNG;20;Charlize
D:\Dev\cpp\OCV\Detection\x64\Debug\images\JL1.PNG;30;Jennifer
D:\Dev\cpp\OCV\Detection\x64\Debug\images\JL2.PNG;30;Jennifer
D:\Dev\cpp\OCV\Detection\x64\Debug\images\JL3.PNG;30;Jennifer
D:\Dev\cpp\OCV\Detection\x64\Debug\images\JL4.PNG;30;Jennifer
D:\Dev\cpp\OCV\Detection\x64\Debug\images\JL5.PNG;30;Jennifer
D:\Dev\cpp\OCV\Detection\x64\Debug\images\JL6.PNG;30;Jennifer
D:\Dev\cpp\OCV\Detection\x64\Debug\images\JL7.PNG;30;Jennifer
D:\Dev\cpp\OCV\Detection\x64\Debug\images\JL8.PNG;30;Jennifer

```

Il y a 7 photos de Charlize Theron. Son indice est 20.  
Il y a 8 photos de Jennifer Lawrence son indice est 30. **5**

Faire le training consiste à charger l'ensemble des images dans un `vector<Mat>` et utiliser la méthode `train` sur un modèle :

```
Ptr<EigenFaceRecognizer> model = EigenFaceRecognizer::create();
for (int i = 0; i < nlabels; i++)
    model->setLabelInfo(i, labelsInfo[i]);
model->train(images, labels);
string saveModelPath = "face-rec-model.txt";
cout << "Saving the trained model to " << saveModelPath << endl;
model->save(saveModelPath);
```

Ensuite, on compare une image (passée en argument sur la ligne de commande) en la passant au modèle :

```
Mat testSample = imread(argv[3], 0);
int predictedLabel = model->predict(testSample);
string result_message =
format("Predicted class = %d / Actual class = %d.",
predictedLabel, testLabel);
cout << result_message << endl;

auto it = labelsInfo.find(predictedLabel);
string name;
if (it != labelsInfo.end())
{
    name = it->second;
    cout << "Name is : " << name << endl;
}
```

Voici la liste des images de tests ; les deux premières sont simples mais la troisième n'est pas ressemblante. **6**

Je compare l'image TEST.PNG au modèle et la sortie est la suivante :

```
Predicted class = 30 / Actual class = -1.
Name is : Jennifer
```

Le modèle fait la prédiction que c'est l'indice 30 qui correspond à Jennifer.

Je compare l'image TEST2.PNG au modèle et la sortie est la suivante :

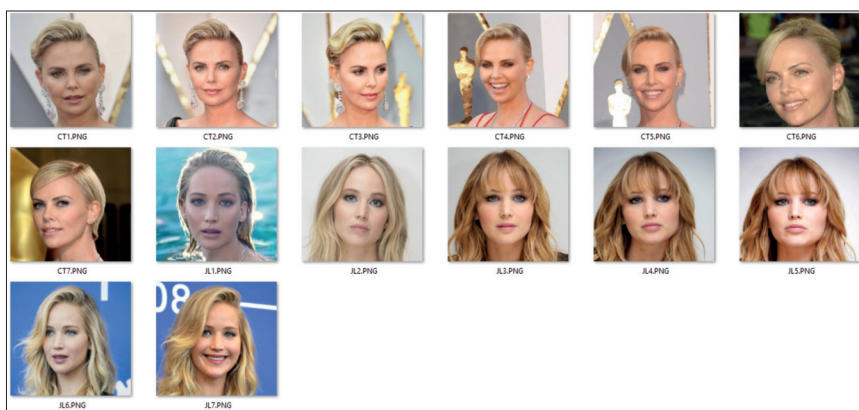
```
Predicted class = 20 / Actual class = -1.
Name is : Charlize
```

Le modèle fait la prédiction que c'est l'indice 20 qui correspond à Charlize.

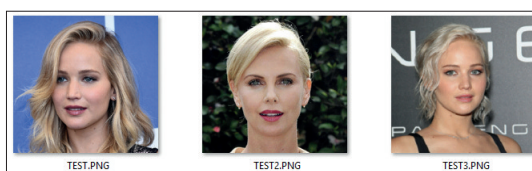
Je fais un dernier essai avec une photo peu prédictible de Jennifer, TEST3.PNG :

```
Predicted class = 30 / Actual class = -1.
Name is : Jennifer
```

Le système a quand même fonctionné. Il a prédit la bonne réponse. Magique ! **7**



**5**



**6**

```
Developer Command Prompt for VS 2017
D:\Dev\cpp\OCVDetection\x64\Debug>OCVDetection.exe config_dior.csv 10 D:\Dev\cpp\OCVDetection\x64\Debug\images_TEST\TEST3.PNG
Processing D:\Dev\cpp\OCVDetection\x64\Debug\images\CT1.PNG
Processing D:\Dev\cpp\OCVDetection\x64\Debug\images\CT2.PNG
Processing D:\Dev\cpp\OCVDetection\x64\Debug\images\CT3.PNG
Processing D:\Dev\cpp\OCVDetection\x64\Debug\images\CT4.PNG
Processing D:\Dev\cpp\OCVDetection\x64\Debug\images\CT5.PNG
Processing D:\Dev\cpp\OCVDetection\x64\Debug\images\CT6.PNG
Processing D:\Dev\cpp\OCVDetection\x64\Debug\images\CT7.PNG
Processing D:\Dev\cpp\OCVDetection\x64\Debug\images\JL1.PNG
Processing D:\Dev\cpp\OCVDetection\x64\Debug\images\JL2.PNG
Processing D:\Dev\cpp\OCVDetection\x64\Debug\images\JL3.PNG
Processing D:\Dev\cpp\OCVDetection\x64\Debug\images\JL4.PNG
Processing D:\Dev\cpp\OCVDetection\x64\Debug\images\JL5.PNG
Processing D:\Dev\cpp\OCVDetection\x64\Debug\images\JL6.PNG
Processing D:\Dev\cpp\OCVDetection\x64\Debug\images\JL7.PNG
Processing D:\Dev\cpp\OCVDetection\x64\Debug\images\JL8.PNG
Saving the trained model to face-rec-model.txt
Predicted class = 30 / Actual class = -1.
Name is : Jennifer
D:\Dev\cpp\OCVDetection\x64\Debug>
```

**7**

L'objet de l'article n'est pas de documenter l'ensemble des fonctionnalités d'OpenCV mais il est possible d'obtenir « une distance » de résultat. En effet, si je passe une photo de ma fille au module, il va me dire que le résultat est plus proche de telle ou telle personne mais avec une distance de plus de 13.000. Je ne sais pas quelle est l'unité à employer. Mais j'ai remarqué qu'à partir de 10.000, la facture de certitude est de 95%.

Pour rendre les choses ludiques, on couple ces fonctionnalités à une caméra et on fait le traitement pour chaque frame de la vidéo.

## Conclusion

OpenCV est une librairie très puissante et passionnante à utiliser. Il y a de nombreuses options que nous n'avons pas couvertes comme la détection d'objets et de formes, les comportements de mouvements, etc.

Si vous êtes intéressé, une seule adresse : <https://opencv.org/>